

The Consensus Problem

- A consensus mechanism is a set of steps that are taken by most or all nodes in a blockchain to agree on a proposed state or value.
- Consensus is a distributed computing concept that has been used in blockchain in order to provide a means of agreeing to a single version of the truth by all peers on the blockchain network.

Characteristics of Consensus

- **Agreement:** mechanism for mutual agreement , All honest nodes decide on the same value
- **Termination:** All honest nodes terminate execution of the consensus process and eventually reach a decision.
- **Validity:** The value agreed upon by all honest nodes must be the same as the initial value proposed by at least one honest node.
- **Fault tolerant:** The consensus algorithm should be able to run in the presence of faulty or malicious nodes (Byzantine nodes).
- **Integrity:** This is a requirement that no node can make the decision more than once in a single consensus cycle

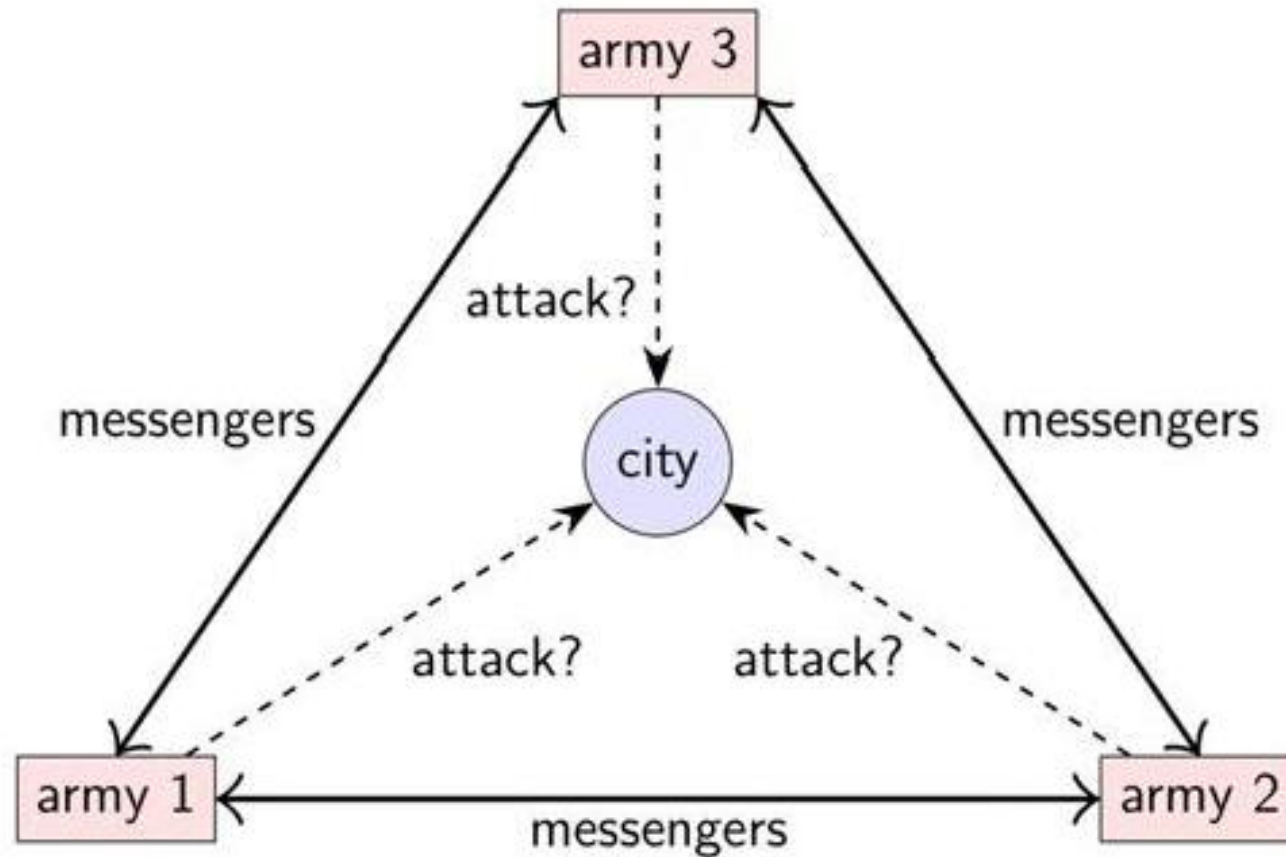
The Consensus Problem

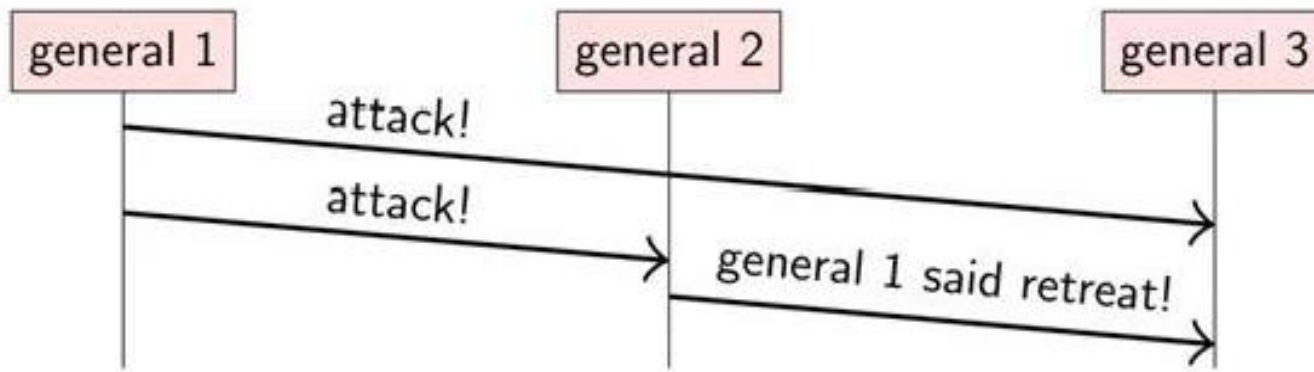
- Consensus is the backbone of a blockchain and, as a result, it provides decentralization of control through an optional process known as **mining**.
- **The choice of the consensus algorithm is also governed by the type of blockchain in use; that is, not all consensus mechanisms are suitable for all types of blockchains. Consensus is a process of agreement between distrusting nodes on the final state of data.**
- **To achieve consensus**, different algorithms are used. It is easy to reach an agreement between two nodes (in client-server systems, for example), but when multiple nodes are participating in a distributed system and they need to agree on a single value, it becomes quite a challenge to achieve consensus. This process of attaining agreement common state or value among multiple nodes despite the failure of some nodes is known as **distributed consensus**.

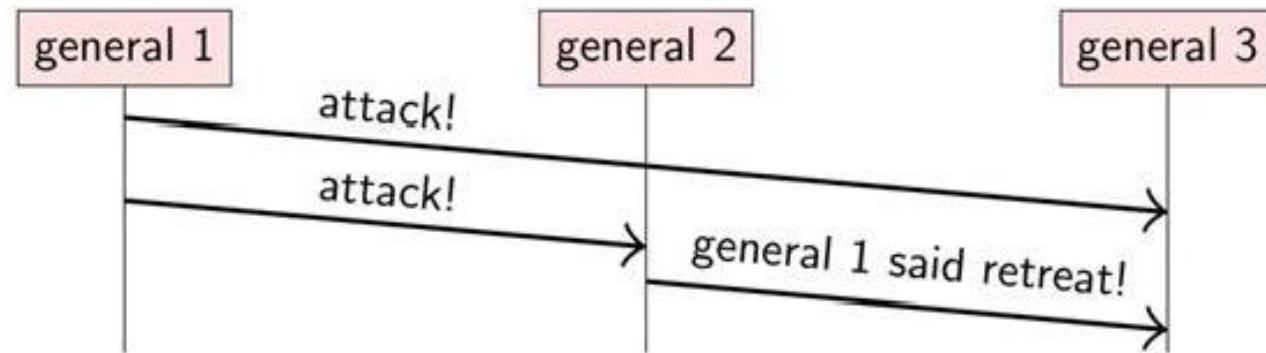
Types of consensus mechanisms

- **Traditional Byzantine Fault Tolerance (BFT)-based:** With no compute-intensive operations, such as partial hash inversion (as in Bitcoin PoW), this method relies on a simple scheme of nodes that are publisher-signed messages. Eventually, when a certain number of messages are received, then an agreement is reached.
- **Leader election-based consensus mechanisms:** This arrangement requires nodes to compete in a leader election lottery, and the node that wins proposes a final value.

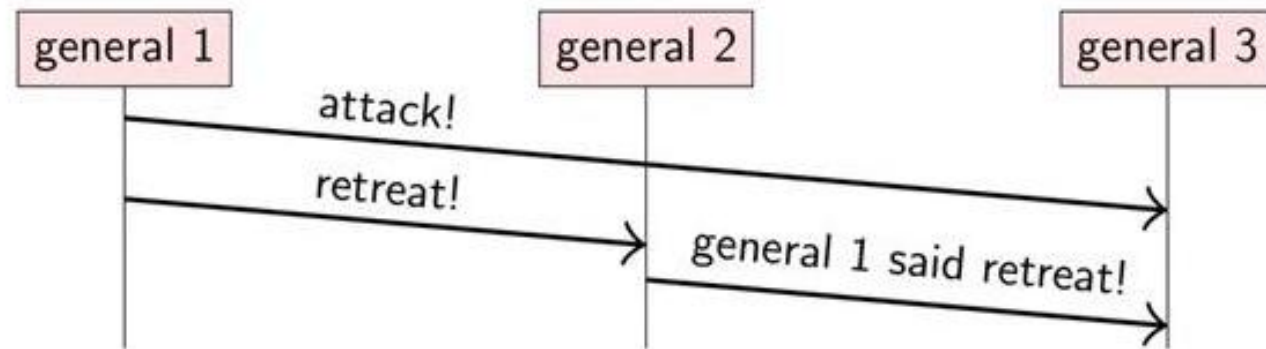
The byzantine General's problem is an illustrative example







From general 3's point of view, this is indistinguishable from:



The byzantine General's problem is an illustrative example

- A number of different armies want to besiege an enemy city. Success requires agreement on a common plan amongst the disbursed army; communication is only by messages.
 - **Compilation:** There may be traitors who send out conflicting messages to sow confusion.
 - **Challenge:** Find an algorithm that ensures the armies come to an agreement and attack at the same time.

Byzantine General's Problem Scenario

- Several armies, each under the command of a general, are camped outside a city which they plan to attack.
- One of the generals will issue the order attack or retreat
- One or several of the generals may be traitors. To win the battle all loyal generals must attack at the same time.
- The traitors will attempt to fool the loyal general so that not all of them attack at the same time.
- To stop the traitor's malicious plan, all generals exchange messages directly with each other.

Byzantine Agreement Problem

- The Byzantine agreement problem was first defined by Lamport. The solution was also given by Lamport first of all under the situation of processor failure.
- According to the concept of Byzantine agreement problem, a source processor is taken to broadcast its initial value to another processor in the system.
- The source processor is selected randomly and sequential ordering is done to choose a source processor.

- For getting the byzantine agreement protocol, it is necessary for all non-faulty processors to be agreed on the same value.
- In some situations, the source processor may be non-faulty. Then its initial value is not used by other processors.

Objective

- The objective of the Byzantine problem is to be able to defend against failures, in which components of a system are in arbitrary ways, i.e not just by stopping or crashing but by processing requests incorrectly, corrupting their local state and producing incorrect or inconsistent outputs.
- The Byzantine failure models real world environments in which computers and networks may behave in unexpected ways due to hardware failures, network congestion and disconnection as well as malicious attacks.

Byzantine General's Problem

- A byzantine fault is an incorrect operation (algorithm) that occurs in a distributed system that can be classified as:
- Omission failure - A failure of not being present such as failing to respond to a request or not receiving a request.
- Execution Failure or lying - A failure due to sending incorrect or inconsistent data, corrupting the local state or responding to requests incorrectly.

Byzantine Algorithm

- A node which can have arbitrary behavior is called byzantine. This includes anything imaginable, eg not sending messages at all, or sending different and wrong messages to different neighbors, or lying about the input value.
- Byzantine Agreement: Finding consensus in a system with byzantine nodes is called byzantine agreement. An algorithm is f -resilient if it still works correctly with f byzantine nodes.

Validity

- Any-Input Validity: The decision value must be the input value of any node.
- Correct input validity : The decision value must be the input value of a correct node.
- All same validity: If all correct nodes start with the same input v , the decision value must be v .
- Median validity
- Synchronous model: each node send message to all other node

How many byzantine nodes

Algorithm 4.9 Byzantine Agreement with $f = 1$.

1: Code for node u , with input value x :

Round 1

2: Send $\text{tuple}(u, x)$ to all other nodes

3: Receive $\text{tuple}(v, y)$ from all other nodes v

4: Store all received $\text{tuple}(v, y)$ in a set S_u

Round 2

5: Send set S_u to all other nodes

6: Receive sets S_v from all nodes v

7: $T =$ set of $\text{tuple}(v, y)$ seen in at least two sets S_v , including own S_u

8: Let $\text{tuple}(v, y) \in T$ be the tuple with the smallest value y

9: Decide on value y

The King Algorithm

Algorithm 4.14 King Algorithm (for $f < n/3$)

```
1:  $x = \text{my input value}$ 
2: for phase = 1 to  $f + 1$  do
    Round 1
3:   Broadcast  $\text{value}(x)$ 
    Round 2
4:   if some  $\text{value}(y)$  at least  $n - f$  times then
5:     Broadcast  $\text{propose}(y)$ 
6:   end if
7:   if some  $\text{propose}(z)$  received more than  $f$  times then
8:      $x = z$ 
9:   end if
    Round 3
10:  Let node  $v_i$  be the predefined king of this phase  $i$ 
11:  The king  $v_i$  broadcasts its current value  $w$ 
12:  if received strictly less than  $n - f$   $\text{propose}(x)$  then
13:     $x = w$ 
14:  end if
15: end for
```

Algorithm 4.21 Asynchronous Byzantine Agreement (Ben-Or,
for $f < n/10$)

```
1:  $x_i \in \{0, 1\}$  / input bit
2:  $r = 1$  / round
3:  $\text{decided} = \text{false}$ 
4: Broadcast propose( $x_i, r$ )
5: repeat
6:   Wait until  $n - f$  propose messages of current round  $r$  arrived

7:   if at least  $n/2 + 3f + 1$  propose messages contain same value
       $x$  then
8:      $x_i = x$ ,  $\text{decided} = \text{true}$ 
9:   else if at least  $n/2 + f + 1$  propose messages contain same
      value  $x$  then
10:     $x_i = x$ 
11:   else
12:     choose  $x_i$  randomly, with  $Pr[x_i = 0] = Pr[x_i = 1] = 1/2$ 
13:   end if
14:    $r = r + 1$ 
15:   Broadcast propose( $x_i, r$ )
16: until  $\text{decided}$  (see Line 8)
17:  $\text{decision} = x_i$ 
```

Assignment No. 1

- Study the white paper of Satoshi Nakamoto and giving your findings.
- <https://www.studocu.com/en-us/document/university-of-the-people/microeconomics/unit-6-assignment-economics/23971303>

Assignment No.2

- Byzantine Agreement: Role Play